

移动应用开发实验报告

封面信息

项目	内容
实验名称	鸿蒙多端应用开发
实验编号	d20301035105、d20301009205
学号	2023210644
姓名	张靖婷
班级	软件 231
日期	2025.5.8
组别	智能健康运动记录平台开发与整合(IHFTPDI)
项目名称	智慧天气应用
技术栈	HarmonyOS (ArkTS)

实验目的

- 掌握 ArkUI 声明式开发方法
- 学会多端响应式布局实现
- 了解传感器数据采集原理
- 理解分布式能力工作原理
- 体验 AI 辅助开发在鸿蒙应用中的应用

实验环境

工具	版本	用途
DevEco Studio	5.0+	鸿蒙应用开发
HarmonyOS SDK	5.0+	鸿蒙开发工具包
模拟器	最新版	多设备模拟测试

工具	版本	用途
AI 辅助工具	TRAE	代码生成与调试

实验步骤

1. 需求分析

1.1 功能需求

天气展示（城市名称、温度、天气状况、空气质量）

多端适配（手机竖屏、平板横屏）

传感器集成（光线传感器、陀螺仪）

1.2 数据模型定义

```
// 天气数据结构
{
    cityName: string;    // 城市名称
    temperature: number; // 温度
    weather: string;     // 天气状况
    aqi: number;         // 空气质量指数
    lightIntensity: number; // 光线强度
}
```

2. 创建鸿蒙项目

2.1 新建项目

打开 DevEco Studio

新建 Empty Ability 项目

选择 ArkTS 语言

命名为 WeatherApp

3. 天气应用开发

3.1 天气首页实现

```
// entry/src/main/ets/pages/Index.ets
```

@Entry

@Component

struct Index {

 @State city: string = "合肥";

 @State temp: number = 24;

 @State weather: string = "晴";

 @State aqi: number = 42;

 build() {

 Column() {

 Text(this.city)

 .fontSize(30)

 .fontWeight(FontWeight.Bold)

 .margin({ top: 60 });

 Text(" ")

 .fontSize(90)

 .margin({ top: 20 });

 Text(`\${this.temp}°C`)

 .fontSize(60)

 .fontWeight(FontWeight.Lighter)

 .margin({ top: 10 });

 Text(this.weather)

 .fontSize(22)

 .margin({ top: 5 });

 Row() {

 Text("空气质量: ")

 Text(`\${this.aqi} 优`)

 .fontColor("#00cc66")

 }

 .margin({ top: 20 });

Text("环境光线: 765.2 lux")

 .margin({ top: 30 });

```

Text("陀螺仪")
  .fontSize(18)
  .fontWeight(FontWeight.Bold)
  .margin({ top: 25 });

Row() {
  Text("X: 0.5")
    .margin({ right: 25 });
  Text("Y: 0.1")
    .margin({ right: 25 });
  Text("Z: 9.7");
}

}

.width('100%')
.height('100%')
.backgroundColor('#f8f9fa')
.justifyContent(FlexAlign.Start)
.alignItems(HorizontalAlign.Center)
}

}

```

4. 多端适配布局

4.1 响应式布局实现

// 断点系统适配

```
@State currentBreakpoint: string = 'md';
```

```
aboutToAppear() {
```

```
  // 获取设备类型
```

```
  let windowWidth = window.getWindow().getWindowProperties().windowWidth;
```

```
  if (windowWidth < 600) {
```

```
    this.currentBreakpoint = 'sm'; // 手机
```

```
  } else if (windowWidth < 840) {
```

```
        this.currentBreakpoint = 'md'; // 平板竖屏
    } else {
        this.currentBreakpoint = 'lg'; // 平板横屏
    }
}

build() {
    if (this.currentBreakpoint === 'sm') {
        // 手机布局

        Column() {
            this.phoneLayout()
        }
    } else {
        // 平板布局

        Row() {
            this.tabletLayout()
        }
    }
}

@Builder phoneLayout() {
    Column() {
        Text(this.cityName).fontSize(24)
        Text('${this.temperature}°C').fontSize(60)
    }
}

@Builder tabletLayout() {
    Column() {
        Text(this.cityName).fontSize(32)
        Text('${this.temperature}°C').fontSize(80)
```

```
}  
    .width('40%')  
}
```

5. 传感器集成

5.1 光线传感器开发

```
import sensor from '@ohos.sensor';  
  
@State lightIntensity: number = 0;  
  
aboutToAppear() {  
    // 注册光线传感器监听  
  
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_LIGHT, (data) => {  
        this.lightIntensity = data.light;  
    });  
}  
  
aboutToDisappear() {  
    // 取消监听  
  
    sensor.off(sensor.SensorType.SENSOR_TYPE_ID_LIGHT);  
}  
  
// UI 展示  
  
Text(`环境光线: ${this.lightIntensity} lux`)  
    .fontSize(16)  
    .margin({ top: 10 })
```

5.2 陀螺仪传感器开发

```
@State rotateX: number = 0;  
  
@State rotateY: number = 0;  
  
@State rotateZ: number = 0;
```

```

aboutToAppear() {
  sensor.on(sensor.SensorType.SENSOR_TYPE_ID_GYROSCOPE, (data) => {
    this.rotateX = data.x;
    this.rotateY = data.y;
    this.rotateZ = data.z;
  });
}

```

// UI 展示

```

Row() {
  Column() { Text(`X 轴: ${this.rotateX.toFixed(2)}`) }
  Column() { Text(`Y 轴: ${this.rotateY.toFixed(2)}`) }
  Column() { Text(`Z 轴: ${this.rotateZ.toFixed(2)}`) }
}

```

6. 分布式数据同步

6.1 分布式数据管理

```

import distributedData from '@ohos.data.distributedData';

// 创建分布式数据库

const KVManager = distributedData.createKVManager({
  bundleName: 'com.example.weather',
  kvStoreType: distributedData.KVStoreType.SINGLE_VERSION
});

// 存储数据

async function saveData(key: string, value: string) {
  const kvStore = await KVManager.getKVStore('weatherStore');
  await kvStore.put(key, value);
}

```

```
// 读取数据

async function getData(key: string) {

    const kvStore = await KVManager.getKVStore('weatherStore');

    return await kvStore.get(key);

}
```

7. AI 辅助开发

7.1 提示词工程应用

应用实践：

- 明确目标： "创建 ArkTS 天气应用页面，声明式 UI，城市温度展示"
- 指定角色： "你是一名鸿蒙 ArkUI 开发专家"
- 明确限制条件： "使用 ArkTS 语言，遵循鸿蒙开发规范"
- 指定输出格式： "提供完整的 ArkTS 代码实现"

7.2 上下文工程应用

应用实践：

- 提供项目结构上下文： "鸿蒙 Empty Ability 项目"
- 提供技术栈上下文： "使用 ArkTS 语言，HarmonyOS 5.0+"
- 提供功能上下文： "需要实现天气展示和多端适配"

实验结果

验证结果

功能	状态	备注
天气信息展示	成功	城市、温度、天气状况、空气质量正常显示
多端适配	成功	手机和平板界面自适应良好
光线传感器	成功	传感器数据实时更新

功能	状态	备注
陀螺仪传感器	成功	三轴数据正常采集
分布式数据同步	成功	跨设备数据同步正常

截图记录

手机端界面



平板端界面



传感器数据展示



技术分析报告

多端适配分析

手机布局特点：垂直排列，紧凑布局，适合单手操作

平板布局特点：横向排列，更宽敞的空间利用，信息展示更丰富

断点系统实现：基于屏幕宽度的响应式设计，自动适配不同设备

传感器集成分析

光线传感器应用场景：自动亮度调节、环境光感知、节能控制

陀螺仪应用场景：设备姿态检测、游戏控制、图像稳定

数据采集准确性：实时性好，精度满足一般应用需求

分布式能力分析

分布式数据管理原理：基于 KV 存储的跨设备数据同步

跨设备同步实现：通过软总线技术实现设备间通信

应用场景：多设备协同、数据共享、无缝切换

问题与解决

问题描述	解决方案	解决结果
模拟器传感器数据不更新	检查传感器权限配置，重启模拟器	传感器数据正常更新
多端适配布局错乱	调整断点阈值，优化布局逻辑	布局自适应正常
分布式数据同步失败	检查网络连接，确认设备在同一网络	数据同步正常

实验总结

实验收获

掌握了 ArkUI 声明式开发方法，理解了声明式 UI 的优势
学会了多端响应式布局实现，能够适配不同尺寸的设备
了解了传感器数据采集原理，能够集成光线传感器和陀螺仪
理解了分布式能力工作原理，能够实现跨设备数据同步
体验了 AI 辅助开发在鸿蒙应用中的应用，提高了开发效率

技术要点

ArkUI 的声明式语法简洁直观，开发效率高
断点系统是实现多端适配的关键技术
传感器 API 使用简单，但需要注意权限管理和资源释放
分布式能力为物联网应用提供了强大的技术支持
AI 辅助开发能够显著提高代码质量和开发效率

考核指标

考核项	评分标准	得分
功能完整性	天气应用功能完整，多端适配正常	25 分
代码质量	代码结构清晰，注释完整	20 分
多端适配	手机/平板界面自适应良好	15 分
传感器集成	传感器数据采集正常	15 分
分布式能力	分布式数据同步功能实现	15 分
实验报告	报告结构完整，内容详实	10 分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	
成绩	

教师签名： _____
日期： _____